

Project 2: N-grams and Naive Bayes by Tien Dao (U20748106) and Dat Tran (U11708956).

To run the test program, keep thing as is in the zip folder after extracting and do: **python HW2_grader.py**

Part 1: N-Gram Detection by Tien Dao:

For Part 1, I implemented an n-gram language model to distinguish a human-like sentence from randomly generated ones. In the training function, I read the training file line by line, tokenized each sentence, and built unigram, bigram, and trigram counts. I also stored the context counts needed to estimate conditional probabilities. In the testing function, I scored each candidate sentence using its average log-probability under the trained model. To make the model more robust, I used smoothing and mapped unseen words to an <UNK> token. Since random sentences should be much less likely under a language model trained on real text, I selected the sentence with the lowest score.

Part 2: Sentiment Classification by Dat Tran:

To solve this problem within the strict <120-second performance limit and specific architectural constraints, I used Python and scikit-learn. For data ingestion, I utilized Python's built-in `json` module to parse the `.jsonl` dataset line-by-line, ensuring memory efficiency rather than loading the entire file at once. To bridge the gap between raw text and machine learning math, I implemented `CountVectorizer` to handle tokenization and English stopword removal simultaneously, converting the text into a sparse matrix of word counts. I selected the `MultinomialNB` algorithm for the classifier because its linear time complexity guarantees rapid training and inference, easily satisfying the time limit while remaining highly effective for baseline NLP tasks. To comply with the isolated function signatures, I deliberately engineered a state-management workaround using global variables, allowing the trained vocabulary and model to persist between the training and testing phases without violating the required API.